

text

Going Secretless with CertManager

Presented by:

Tim & Ash – CertManager Maintainers, CyberArk (formerly Venafi)

Event: KubeCon

Topic: Secretless Certificate Issuance and Provisioning in Kubernetes

1. Introduction

CertManager is a Kubernetes-native solution for managing **X.509 certificates** across workloads. It automates certificate issuance, renewal, and storage using the Kubernetes API, acting as an intermediary between applications and Certificate Authorities (CAs).

Applications leverage these certificates for **server and client authentication**, ensuring encrypted and verified communication across services. Beyond its technical role, CertManager represents a thriving open-source community focused on **PKI, security automation, and controller design** for Kubernetes.

At this year's KubeCon, the CertManager project proudly announced its graduation within the **Cloud Native Computing Foundation (CNCF)** — a milestone recognizing the maturity, quality, and community strength of the project.

2. From Secrets to Secretless

2.1 What Does “Secretless” Mean?

In a secretless system, **no shared secrets** (such as passwords, tokens, or API keys) are transmitted or stored across machines or networks.

Instead, authentication is achieved through **locally generated, non-transferable credentials**.

Examples:

-  Username/password stored in Kubernetes Secrets → *Not secretless*
-  Shared one-time tokens → *Not secretless*
-  Locally generated keys used for challenge/response → *Secretless*

The goal is to make the entire certificate issuance process **fully secretless**, minimizing the attack surface and avoiding secret management overhead.

(Note: JWTs are “almost secretless.” They inherit many good properties but still involve transmitting a token value that could be intercepted.)

3. Certificate Issuance Workflow

CertManager supports many issuers — including **Venafi**, **HashiCorp Vault**, **AWS PCA**, **Google CA Service**, and **Let's Encrypt**.

The general issuance process involves:

1. **Creating a CSR (Certificate Signing Request)**
2. **Sending the CSR to a CA** for signing
3. **Receiving and storing** the signed certificate

The challenge lies in authenticating to the CA securely — ensuring that only authorized workloads can request certificates for specific domains or identities.

4. Traditional Secret-Based Authentication

The simplest approach uses a **static secret** (e.g., Vault token) stored in a Kubernetes Secret. While easy to implement, this approach has major drawbacks:

- **Manual setup** and difficult GitOps integration
- **High-value static tokens** that are attractive to attackers
- **Lack of automatic rotation**

Static secrets violate the secretless principle and increase operational risk.

5. Workload Identity: A Better Way

A more secure alternative uses **workload identity** — an authentication mechanism derived from the runtime environment itself.

In Kubernetes, this is typically achieved through **service account tokens**:

- Each pod has a service account and can request a **signed JWT** from the API server.
- CertManager can use this JWT to authenticate with the CA.
- The CA verifies the JWT using the Kubernetes cluster's **public JWKS keys** via **OpenID Connect**.

Benefits:

- No static secrets stored or shared
- JWTs are **time-limited and auto-rotated**
- Workload-specific authentication

Limitations:

- JWTs must still be transmitted over the wire, making them not *fully* secretless.

- If intercepted, they can be reused until expiration.
-

6. Toward Full Secretlessness: Using X.509 for Authentication

A more robust approach is to use **X.509 client certificates (mTLS)** as the authentication mechanism itself.

With **mutual TLS**, workloads use a private key they already possess to prove their identity to the CA — eliminating any need to send a secret.

Advantages:

- No secret transmission or sharing
- Cryptographically verifiable identities
- Fully secretless interaction

Challenges:

- Few CAs currently support certificate-based bootstrapping
 - Typically requires an existing identity provider (e.g., **SPIFFE/SPIRE**)
-

7. Secretless Provisioning: The CSI Driver

Even after secure issuance, the default CertManager workflow stores issued certificates and keys in a **Kubernetes Secret**.

To eliminate this dependency, CertManager provides a **CSI Driver Add-on**, enabling **in-memory certificate provisioning** directly to pods.

How it Works:

- Pods define a **volume** using the `cert-manager-csi-driver` plugin.
- The CSI driver requests certificates on behalf of the pod.
- The resulting private key and certificate are mounted **in-memory** inside the container.
- No data is stored in Kubernetes Secrets or persistent storage.

Benefits:

- Certificates never leave the node memory.
- No cluster-wide secret access.
- Strong isolation and minimized attack surface.

Limitations:

- Suitable for **short-lived, dynamic workloads**.
 - Every pod restart triggers a new certificate issuance (hence best with private PKI, not Let's Encrypt).
 - Avoids rate limits by relying on internal CA solutions.
-

8. Demonstration Summary

The demo showcased:

1. A Kubernetes Kind cluster running CertManager, CSI Driver, and Vault.
2. Vault configured as a PKI backend with root CA and OIDC trust configuration.
3. CertManager issuer configured to authenticate via service account tokens.
4. A workload using the CSI driver to mount an in-memory certificate volume.

Result:

- The pod successfully received a valid certificate issued by Vault.
 - No Kubernetes Secret resources were created.
 - All operations remained local to the node and in memory.
-

9. Key Takeaways

Area	Traditional	Secretless Approach
Authentication	Static tokens, secrets	Workload identity (JWT, SPIFFE)
Storage	Kubernetes Secrets	In-memory volumes (CSI Driver)
Rotation	Manual / external	Automatic, short-lived
Exposure Risk	High	Minimal
Best suited for	Long-lived apps, public CA (e.g., Let's Encrypt)	Short-lived workloads, private PKI

10. Limitations & Future Work

- **Revocation** is not currently supported; instead, CertManager encourages **short certificate lifetimes** (e.g., one hour).
 - **StatefulSets** requiring identical certificates may still rely on shared Secrets.
 - Broader CA support for **mTLS-based issuance** would enable truly secretless operation across all use cases.
 - Upcoming work includes tighter integration with **SPIFFE/SPIRE** for identity federation and cross-cluster authentication.
-

11. Community & Contribution

CertManager welcomes contributions of all kinds — not just code:

- Improving documentation and examples
- Answering community questions on Slack
- Testing integrations and writing tutorials

Resources:

-  cert-manager.io
 -  CNCF Slack (`#cert-manager`)
 -  Contribution Guide & Meeting Links available via QR code
-

12. Conclusion

CertManager's evolution toward **secretless certificate management** represents a major step in Kubernetes security automation.

Through **workload identities**, **CSI Driver in-memory provisioning**, and **community-driven innovation**, the project continues to simplify and harden PKI management for cloud-native environments.

“Avoid static secrets, embrace ephemeral identity.”

– The CertManager Team